

**SIMATS ENGINEERING**  
**ASSESSMENT TOOL 3: Reverse Engineering Task**

|                     |                                           |                     |         |
|---------------------|-------------------------------------------|---------------------|---------|
| <b>COURSE NAME:</b> | Cloud Computing and<br>Big Data Analytics | <b>COURSE CODE:</b> | CSA1516 |
|---------------------|-------------------------------------------|---------------------|---------|

---

**COURSE OUTCOME COVERED**

**CO4:** *Analyze big data characteristics and challenges across domains including security and application aspects.*  
(BL4)

Dave's Taxonomy: Articulation (Level 4)  
Question Count: 5

**Total Marks: 30**

---

**Code of Conduct**

I \_\_Y SRAVAN KUMAR (192472289)\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: \_\_\_\_SRAVAN Y\_\_\_\_

**Assessment Tasks**

**Reverse Engineering Task**

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy,

encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

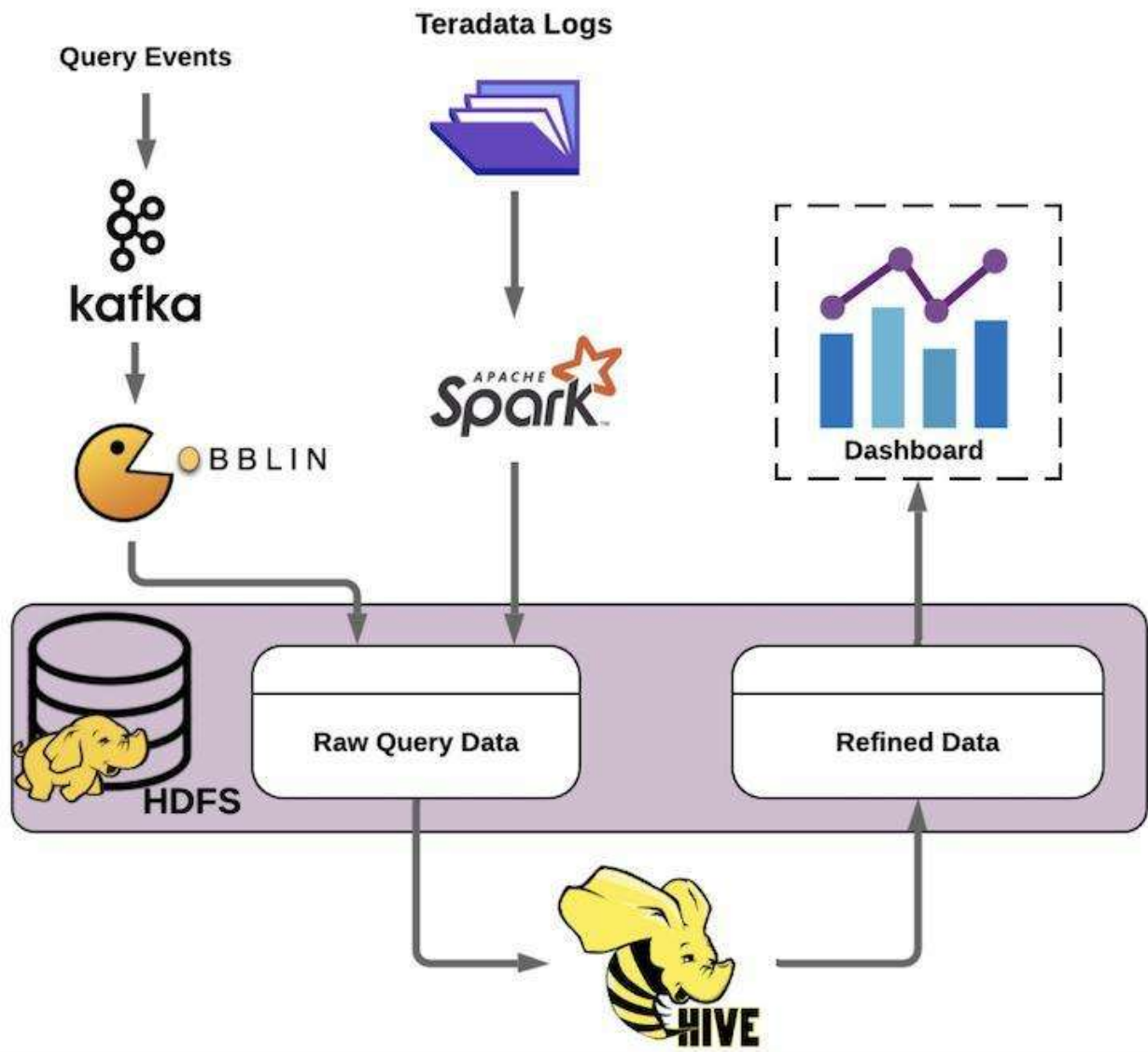
4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.
5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

#### Reverse Engineering Task Rubric

| Criteria                      | Weightage | Level 4 - Exemplary                                           | Level 3 - Proficient      | Level 2 - Developing  | Level 1 - Beginning    |
|-------------------------------|-----------|---------------------------------------------------------------|---------------------------|-----------------------|------------------------|
| Inference Accuracy            | 25%       | Infers system components and flow with high accuracy          | Mostly accurate inference | Basic inference       | Weak inference         |
| Understanding of Architecture | 25%       | Shows clear understanding of underlying big data architecture | Good understanding        | Partial understanding | Poor understanding     |
| Use of Technical Evidence     | 20%       | Uses strong reasoning and technical evidence                  | Reasonable evidence       | Limited evidence      | No meaningful evidence |
| Depth of Analysis             | 15%       | Analysis is deep and insightful                               | Reasonably deep           | Basic depth           | Superficial analysis   |
| Clarity                       | 15%       | Clear and organized explanation                               | Generally clear           | Somewhat unclear      | Poorly presented       |

**Total Marks Awarded:**

## 1. Big Data Platform – Clicks, Mobile Interactions & Transactions



### Problem Understanding

The platform collects **billions of website clicks, mobile interactions, and customer transactions every day**. Such a large amount of data requires a distributed architecture that can handle high volume, high velocity, and different data formats.

### Probable Architecture

**Data Sources → Kafka → Distributed Storage → Spark → Analytics/ML → BI Dashboards**

#### 1. Distributed Storage

A distributed storage system such as **Hadoop HDFS** or a **Cloud Data Lake** is likely used.

- **Structured data:** Customer transactions, payments, orders, and account information.
- **Semi-structured data:** Website click logs, mobile events, JSON logs, and application events.
- Data is distributed across multiple machines rather than stored on one server.
- Replication is likely used to prevent data loss if a machine fails.

## 2. Data Ingestion

**Apache Kafka** is a probable ingestion framework.

Kafka can continuously collect:

- Website clicks
- Mobile application events
- Transactions
- Customer activities

Kafka topics can be separated according to event type, such as:

**clicks → mobile\_events → transactions → payments**

## 3. Batch Processing

**Apache Spark** is likely used for large-scale batch processing.

Spark can:

- Clean data
- Remove duplicates
- Transform data
- Join datasets
- Calculate customer statistics
- Prepare data for machine learning

## 4. Data Organization and Indexing

Structured transaction data can be stored in **tables or warehouse systems**, while semi-structured clickstream data can be stored in a **Data Lake**.

Data may be partitioned by:

- Date
- Region
- Customer
- Event type

For example:

## 2026 → India → Website Clicks

Indexes and metadata catalogs can then help analysts find required datasets quickly.

## 5. Real-Time Processing

For real-time analytics, the platform could use:

- **Spark Structured Streaming**
- **Apache Flink**
- Kafka Streams

These systems can calculate live metrics such as:

- Active users
- Current transactions
- Click rates
- Conversion rates

## 6. Scalability and Partitioning

The system probably uses **horizontal scaling**, where more servers are added when data volume increases.

Kafka data can be divided into **partitions**, allowing multiple consumers to process data simultaneously.

Distributed storage can also divide large datasets across multiple nodes.

## 7. Dashboard and BI Connection

Processed data can be stored in an **analytics warehouse or serving database**.

Tools such as:

- Power BI
- Tableau
- Looker

can connect to this analytics layer and generate dashboards.

## Expected Architecture

**Website + Mobile + Transactions**

↓

**Kafka**

↓

**HDFS / Cloud Data Lake**

↓

**Spark Batch + Spark Streaming/Flink**

↓

**Analytics / ML Layer**

↓

**Data Warehouse**

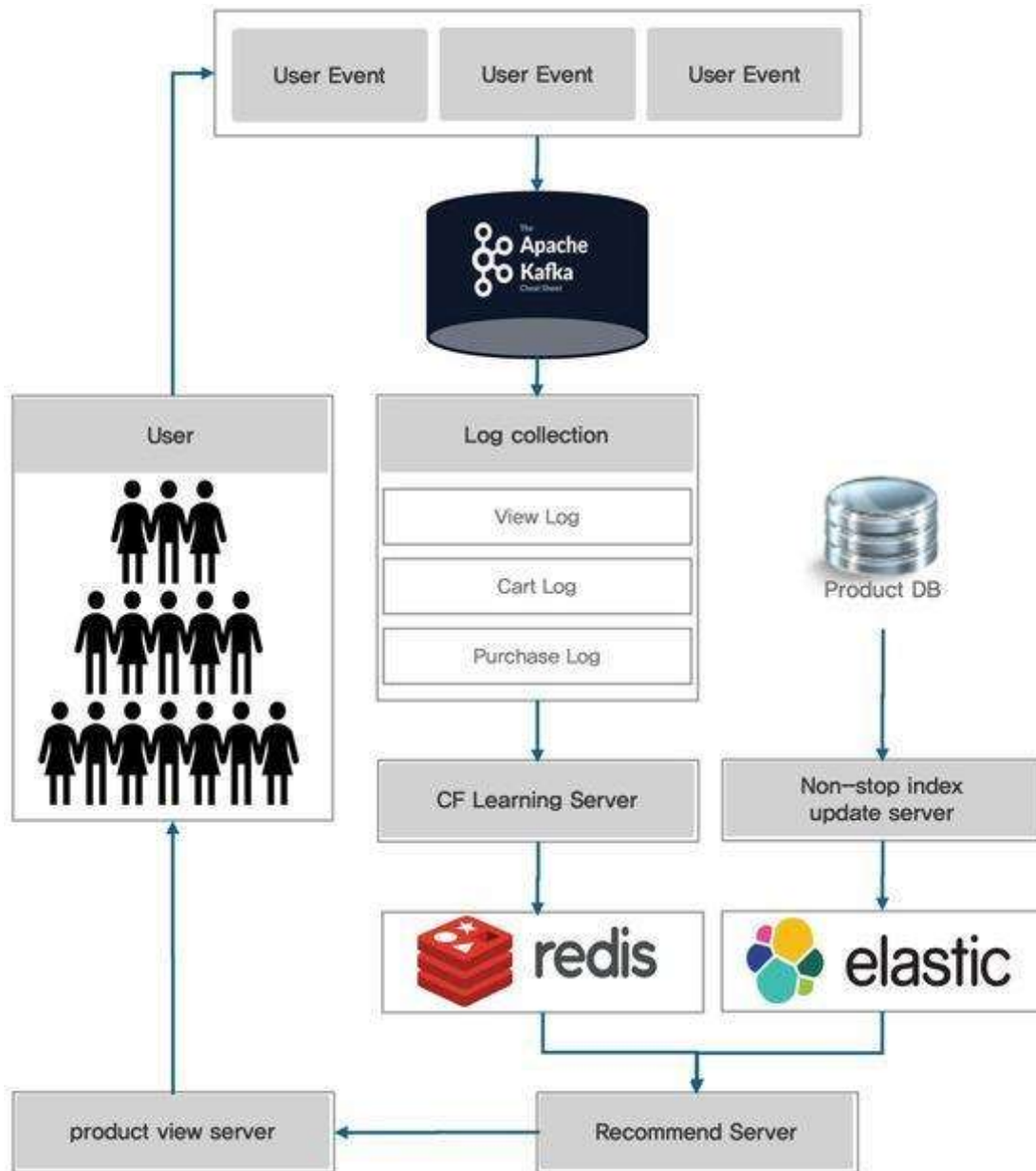
↓

**Power BI / Tableau**

## **Main Design Decisions**

The architecture is likely designed for **high scalability, fault tolerance, parallel processing, real-time analytics, and easy business reporting.**

## 2. E-Commerce – Recommendation & Personalization



### Problem Understanding

The e-commerce platform tracks **cart additions, abandoned checkouts, product views, and payment behavior** across different regions. The main requirement is to understand customer behavior and provide real-time personalization.

### Probable Architecture

**Customer Activity → Kafka → Processing → Customer Profile → Recommendation Engine → E-Commerce Application**

## 1. Event Streaming

**Apache Kafka** is likely used to collect real-time events such as:

- Product views
- Cart additions
- Checkout attempts
- Abandoned carts
- Payments
- Searches
- Product clicks

Different Kafka topics can be used for different event categories.

## 2. Customer Profiling Database

A distributed NoSQL database such as:

- **Cassandra**
- MongoDB
- DynamoDB

could maintain customer profiles.

The profile may contain:

- Customer ID
- Recently viewed products
- Previous purchases
- Cart items
- Preferred categories
- Location
- Browsing behavior

## 3. Session Data Synchronization

Customer session information can be processed in real time.

For example:

**Customer views phone → adds phone to cart → leaves checkout**

The system can immediately update the customer profile and identify an abandoned cart.

## 4. Caching

A caching system such as **Redis** can store frequently required information.

It can provide:

- Fast customer profile access
- Recently viewed products
- Recommendation results
- Shopping cart information

This reduces the response time of the application.

## 5. Distributed Query Engine

Large historical datasets can be queried using systems such as:

- **Trino**
- Presto
- Spark SQL

These can analyze data stored in distributed storage.

## 6. Machine Learning Workflow

The recommendation pipeline may follow:

**Customer Data → Data Cleaning → Feature Engineering → ML Model → Product Ranking → Recommendation**

Possible techniques include:

- Collaborative Filtering
- Content-Based Filtering
- Matrix Factorization
- Deep Learning

## 7. Structured and Semi-Structured Data

**Structured:**

- Orders
- Payments
- Customer details
- Product information

**Semi-structured:**

- Clickstream logs
- JSON events
- Browsing behavior
- Application logs

These datasets can be joined using identifiers such as **Customer ID, Product ID, and Session ID**.

## 8. Scalability and Fault Tolerance

The platform probably uses:

- Kafka partitions
- Distributed databases
- Replication
- Multiple processing nodes
- Automatic recovery
- Load balancing

This allows the system to continue operating even when some components fail.

## 9. Dashboard and KPI Monitoring

Business dashboards can display:

- Conversion rate
- Cart abandonment rate
- Sales by region
- Product popularity
- Customer activity
- Revenue
- Recommendation performance

## Expected Architecture

### Customer Activities

↓

### Kafka

↓

### Spark/Flink

↓

### Customer Profile Database + Data Lake

↓

### ML Recommendation Engine

↓

### Redis Cache

↓

## Personalized E-Commerce Experience

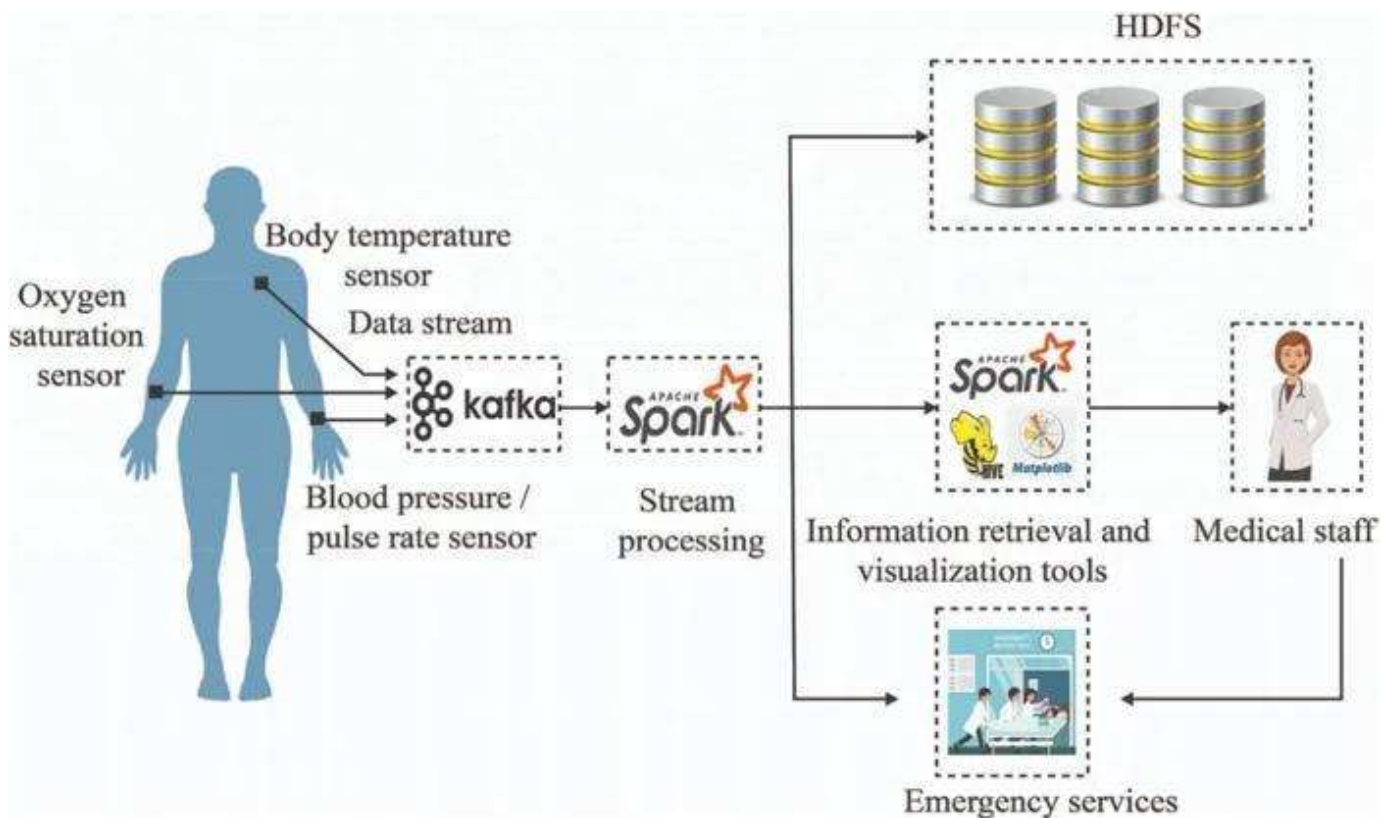
At the same time:

Processed Data → Warehouse → Power BI/Tableau → KPI Dashboard

## Main Design Decision

The architecture focuses on **real-time personalization**, **low latency**, **scalable storage**, **fault tolerance**, and **customer behavior analysis**.

## 3. Healthcare – Patient, Images, Sensors & Prescriptions



## Problem Understanding

The healthcare system handles different types of sensitive data including **patient histories**, **diagnostic images**, **wearable sensor streams**, and **prescription records**.

Because healthcare data is both highly sensitive and diverse, a hybrid architecture is likely required.

## Probable Architecture

**Hospitals + Labs + Wearables + Insurance → Integration Layer → Hybrid Storage → Processing → ML → Clinical Analytics**

### 1. Hybrid Database Architecture

Different storage systems can be used for different data.

#### Relational Database:

- Patient records
- Prescription information
- Billing
- Laboratory results
- Appointments

#### Object/Data Lake Storage:

- Diagnostic images
- Medical documents
- Large files

#### NoSQL Database:

- Wearable sensor readings
- Real-time patient measurements

### 2. Data Integration

Hospitals, laboratories, and insurance systems may use integration tools such as:

- ETL/ELT pipelines
- Apache NiFi
- APIs
- Healthcare interoperability standards

Healthcare standards such as **HL7** and **FHIR** can help systems exchange patient information.

### 3. Streaming

Wearable devices continuously generate information such as:

- Heart rate
- Blood pressure
- Oxygen level

- Activity
- Temperature

A streaming platform such as **Kafka** can collect these events.

**Kafka → Spark Streaming/Flink → Monitoring System**

## 4. Machine Learning

Machine learning can analyze patient information to identify risk patterns.

Possible applications:

- Disease prediction
- Patient risk scoring
- Readmission prediction
- Abnormal vital-sign detection
- Population health analysis

## 5. Privacy and Security

Healthcare systems require strong security.

Likely measures include:

- Encryption
- Authentication
- Role-Based Access Control
- Data masking
- Anonymization
- Audit logs

Access can be restricted according to roles such as:

**Doctor → Patient clinical data**

**Billing staff → Billing information**

## 6. Compliance

The platform would need to follow applicable healthcare privacy and security requirements. The exact regulations depend on the country and organization.

## 7. Clinical Analytics Pipeline

**Patient Data → Integration → Data Lake/Database → Data Processing → ML/Analytics → Clinical Decision Support**

Doctors could receive:

- Patient risk scores
- Disease indicators
- Abnormal readings
- Historical patient trends

## 8. Population Health Studies

Large-scale historical data can be processed using **Apache Spark** to identify trends across populations.

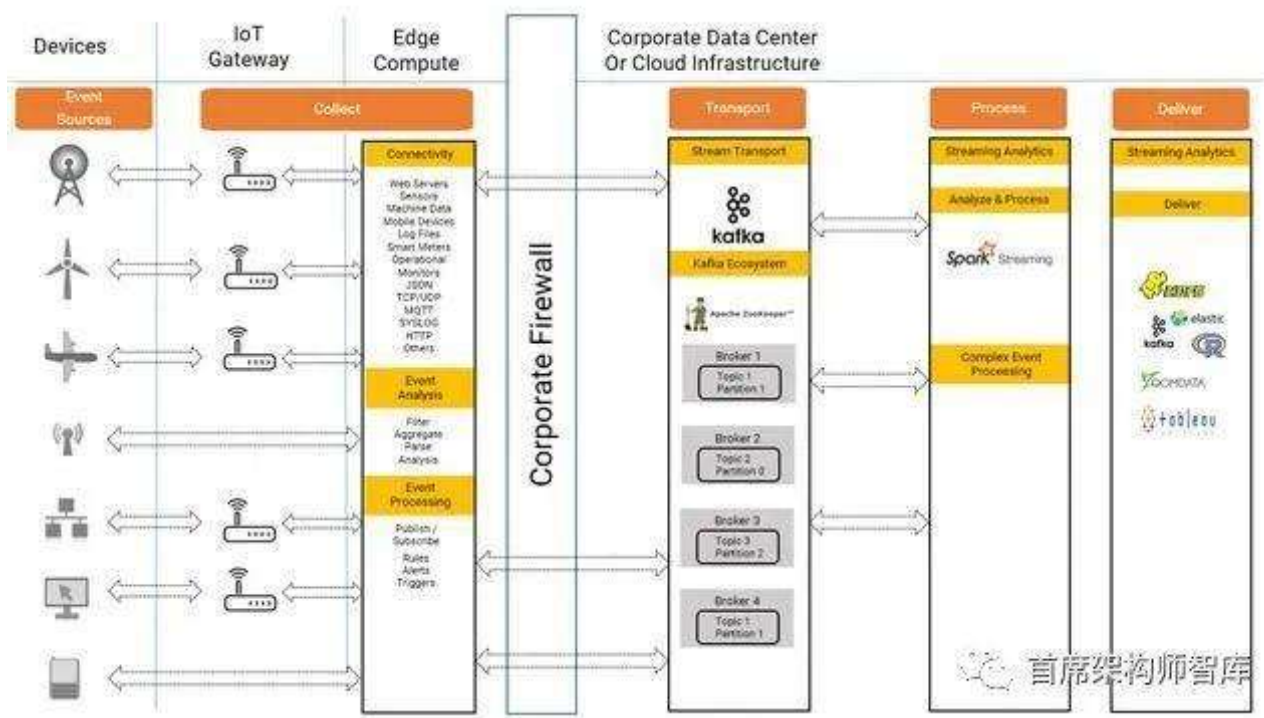
For example:

- Disease patterns
- Regional health trends
- Treatment outcomes
- Risk factors

## Main Design Decision

The system needs to balance **high data volume, different data formats, real-time monitoring, security, privacy, and reliable clinical analysis.**

## 4. Smart City – Traffic, CCTV, Weather & Transport



### Problem Understanding

A smart city continuously collects data from **traffic sensors, CCTV systems, weather services, and public transport**. The system needs to detect current conditions and predict congestion.

### Probable Architecture

**Sensors/CCTV/Weather/Transport → Edge → Kafka → Stream Processing → Storage → Analytics → Dashboard**

#### 1. Edge Computing

Edge devices can process some information close to where it is generated.

For example:

**Traffic Camera → Edge Device → Detect Vehicle Count → Send Result**

Instead of sending every raw video frame to the central system, the edge device can send useful metadata such as:

- Vehicle count
- Traffic density
- Location
- Timestamp

This reduces network bandwidth and latency.

## 2. Distributed Messaging

**Apache Kafka** is a likely messaging system.

It can collect:

- Traffic events
- Weather updates
- Transport events
- CCTV metadata

## 3. Centralized Storage

A **Cloud Data Lake** or **HDFS** can store historical city data.

The data can be organized by:

**Date + Location + Sensor Type**

## 4. Geospatial Analytics

The platform needs location-based analysis.

Possible technologies include:

- PostGIS
- GeoMesa
- Spark Spatial/GeoSpark

These can analyze relationships between:

- Roads
- Traffic sensors
- Bus routes
- Accident locations
- Weather conditions

## 5. Time-Series Database

Traffic and sensor readings are time-dependent.

Possible databases include:

- InfluxDB
- TimescaleDB

They can efficiently answer queries such as:

**“What was the traffic level on this road during the last 30 minutes?”**

## 6. Congestion Prediction

Machine learning models can use:

- Historical traffic
- Current traffic
- Weather
- Time of day
- Day of week
- Public transport activity

to predict future congestion.

## 7. Batch + Real-Time Analytics

**Real-time analytics** can identify:

- Current traffic congestion
- Accidents
- Transport delays

**Batch analytics** can identify:

- Long-term traffic patterns
- Road usage
- Infrastructure requirements
- Urban development trends

## 8. Security

Security measures may include:

- Authentication
- RBAC
- Encryption
- Network security
- Audit logs
- Secure APIs

Access to CCTV and citizen-related information should be restricted.

## 9. Visualization

Operational dashboards can display:

- Live traffic maps
- Congestion levels
- Bus locations
- Weather conditions
- Traffic alerts

Possible tools:

- Power BI
- Tableau
- Grafana

## Expected Architecture

**Traffic Sensors + CCTV + Weather + Transport**

↓

**Edge Processing**

↓

**Kafka**

↓

**Spark/Flink**

↓

**Time-Series DB + Data Lake**

↓

**Geospatial + ML Analytics**

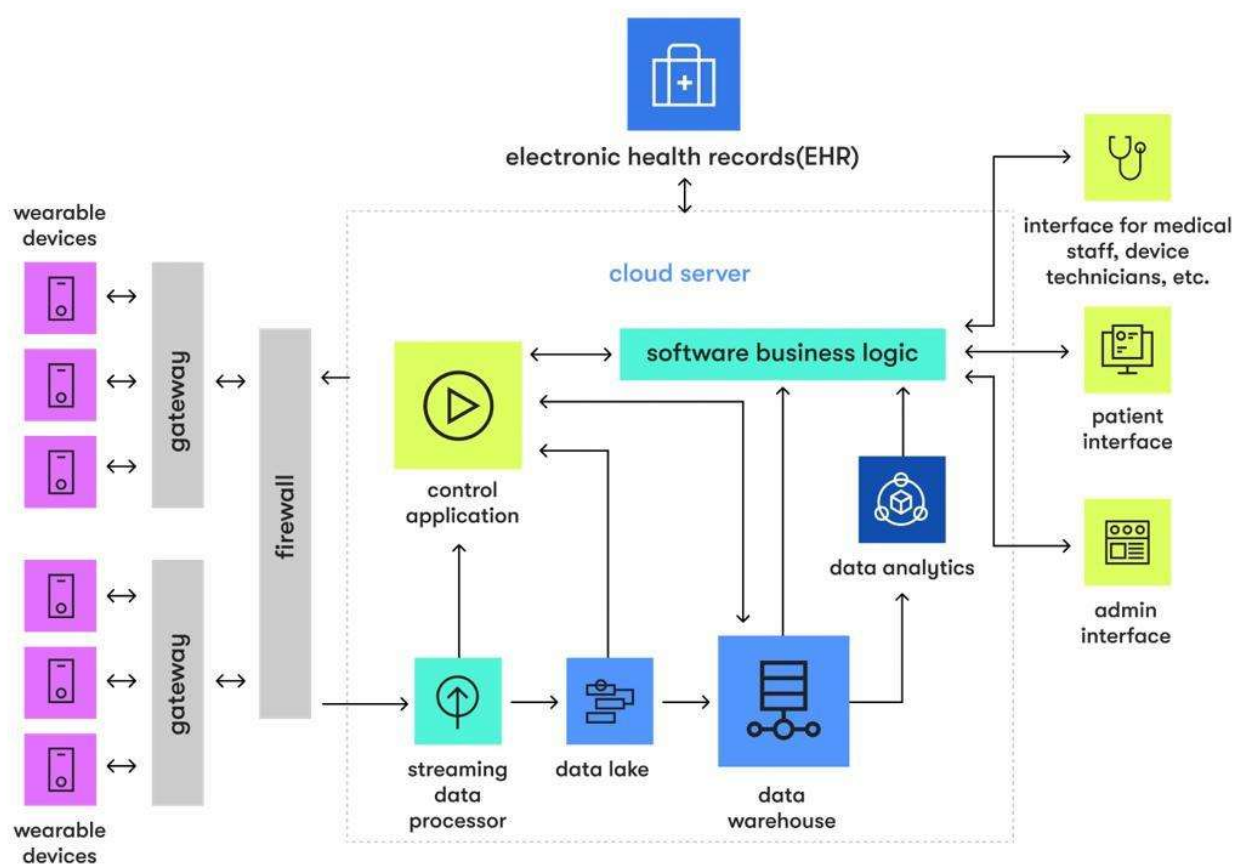
↓

**Live Dashboard**

## Main Design Decision

The architecture focuses on **low latency, location-aware processing, continuous streaming, scalability, and real-time city operations.**

## 5. Healthcare Analytics – Records, Wearables, Labs & Appointments



### Problem Understanding

This system combines **patient records, wearable readings, laboratory reports, and appointment histories**. It needs to provide predictive healthcare analytics while protecting sensitive medical information.

### Probable Architecture

**Hospitals + Clinics + Wearables → ETL/Integration → Hybrid Storage → Processing → ML → Healthcare Analytics**

#### 1. Hybrid Storage

**Relational Database:**

- Patient records
- Appointments
- Lab results

- Prescriptions

#### **Data Lake/Object Storage:**

- Medical documents
- Reports
- Images
- Historical datasets

#### **NoSQL/Time-Series Database:**

- Wearable readings
- Continuous health measurements

## **2. ETL Pipeline**

The ETL process can be:

**Extract → Transform → Load**

### *Extract*

Collect data from:

- Hospitals
- Clinics
- Wearable devices
- Laboratory systems

### *Transform*

- Remove duplicates
- Clean missing values
- Standardize formats
- Validate records

### *Load*

Store the processed data in the appropriate database or data lake.

## **3. Interoperability**

Healthcare systems can use standards such as:

- **HL7**
- **FHIR**

- APIs

These help different hospitals and applications exchange healthcare information.

## 4. Real-Time Monitoring

Wearable devices continuously generate data.

**Wearable → Kafka → Stream Processing → Monitoring**

The system can detect abnormal readings and generate alerts.

## 5. Predictive Health Analytics

Machine learning can use historical and current data to calculate health risks.

Possible outputs:

- Disease risk score
- Patient deterioration risk
- Readmission risk
- Abnormal health pattern
- Treatment-related predictions

## 6. Distributed Processing

Population-level healthcare datasets can become extremely large.

**Apache Spark** can distribute processing across multiple machines.

It can analyze:

- Millions of patient records
- Disease trends
- Treatment outcomes
- Regional health patterns
- Risk factors

## 7. Data Governance and Security

Because medical data is sensitive, the system may use:

- Encryption at rest
- Encryption during transmission
- Authentication
- RBAC

- Data masking
- Anonymization
- Audit logs
- Data retention policies

## 8. Machine Learning

The ML workflow can be:

**Patient Data → Data Cleaning → Feature Engineering → ML Model → Risk Score → Clinical Recommendation**

Models could include:

- Logistic Regression
- Random Forest
- XGBoost
- Neural Networks

## 9. Diagnosis and Treatment Support

The system can provide decision-support information such as:

**Patient information → Risk analysis → High-risk score → Doctor reviews result**

The ML output should support clinical decisions rather than independently replace qualified medical professionals.

## Expected Architecture

**Patient Records + Wearables + Labs + Appointments**

↓

**ETL + HL7/FHIR/API Integration**

↓

**Hybrid Storage**

↓

**Spark + Kafka/Flink**

↓

**Machine Learning**

↓

**Risk Scoring / Analytics**

↓

**Healthcare Dashboard**

## Main Design Decision

The architecture is designed around **data privacy, interoperability, real-time monitoring, distributed processing, predictive analytics, and secure healthcare data management.**